# AI Metro: A Crowd-Aware Metro Route Planner Using A* Search

Ajay Prasath P
CSE
Rajalakshmi Engineering College
Thandalam
ajayprasath.p.2024.cse@rajalakshmi.edu.in

Abirami L
CSE
Rajalakshmi Engineering College
Thandalam
abirami.l.2024.cse@rajalakshmi.edu.in

***Abstract*—Conventional metro route planners typically minimise distance or travel time and therefore overlook passenger-centred factors such as station crowding, service availability and the inconvenience of line changes. This paper presents AI Metro, a crowd-aware route planner that applies A* search to a weighted metro network graph in which stations are nodes and bidirectional connections are edges. Each edge carries a dynamic cost composed of a fare-based travel cost, a time cost, a crowd penalty derived from Low, Medium and High crowd levels, and a transfer penalty applied at line changes. User preferences such as avoiding crowds, minimising time or minimising transfers re-weight these components. Stations and connections that are closed, under maintenance or disrupted are removed from the search graph before the search begins, so they are excluded rather than merely penalised. An admissible straight-line heuristic, scaled to the same units as the path cost, guides A* and preserves optimality. The system also returns alternative routes together with total travel time, fare, crowd summary and number of transfers. The A* implementation is explicit and exposes its open and closed sets and f, g and h values, which makes the search explainable. A prototype with a simulated sample network, simulated crowd and availability data, and an interactive web interface demonstrates how the recommended route changes with crowding, closures and passenger preferences.**

***Index Terms*—A* search, metro route planning, crowd-aware routing, dynamic edge cost, admissible heuristic, station availability, transfer penalty, explainable AI**

## I. INTRODUCTION

Urban metro systems carry large numbers of passengers every day, and journey planners are often the first tool a commuter uses. Most planners model the network as a graph and apply a shortest-path algorithm to distance or scheduled travel time (Dijkstra, 1959; Bast et al., 2016). This is simple and fast, but it describes the network on an ideal day. In practice a platform may be packed during peak hours, a station may be closed for maintenance, and the route that looks fastest on paper may need several inconvenient line changes.

Three factors are commonly missing from conventional planners. The first is crowding. Studies of rail travel report that passengers treat crowded conditions as a significant extra travel-time penalty (Wardman and Whelan, 2011), so a crowded route can be less attractive than a slightly longer, quieter one. The second is availability. Closures, maintenance and disruptions make some stations and connections unusable, and a planner that only penalises them may still recommend an invalid route. The third is the transfer burden. Each line change adds walking, waiting and uncertainty, yet it is rarely counted as a cost.

Artificial Intelligence offers a natural way to include these factors. Informed search algorithms such as A* order their exploration by $f(n) = g(n) + h(n)$, where $g(n)$ is the cost already incurred and $h(n)$ estimates the remaining cost, and they return an optimal path when the heuristic never over-estimates (Hart, Nilsson and Raphael, 1968). A* works with any non-negative cost function, so it can balance several quantities instead of a single one.

This paper presents AI Metro, a crowd-aware metro route planner built on A* search. The metro network is a weighted graph in which stations are nodes and bidirectional connections are edges. The cost of each edge is calculated dynamically as

edge_cost = travel_cost + time_cost + crowd_penalty + transfer_penalty,

using current crowd levels (Low, Medium or High) and the availability status of stations and connections. Unavailable elements are removed from the graph before the search starts, so they are excluded entirely rather than just penalised. User preferences such as avoiding crowds, minimising time or minimising transfers re-weight the cost components without changing the algorithm. Besides the best route, the system returns alternative routes with total time, fare, crowd summary and number of transfers.

The contributions of this work are as follows:

- A cost model that combines fare, travel time, crowd level and transfer penalty into one edge cost for A* search.
- Dynamic exclusion of closed, under-maintenance and disrupted stations and connections, which can be toggled at run time without restarting the application.
- An admissible heuristic based on straight-line distance, scaled to the units of the path cost so that the route found remains optimal.
- A transparent A* implementation that exposes its open and closed sets and its f, g and h values, which makes each recommendation explainable.
- An interactive prototype that uses a simulated sample network, simulated crowd and availability data, and a web interface to show how the recommended route changes with crowding, closures and preferences.

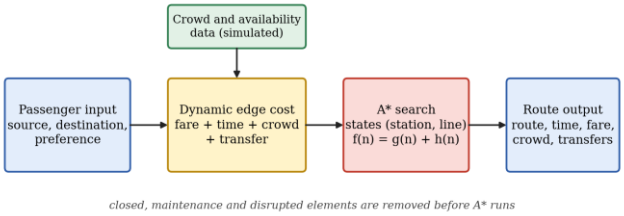



**Fig. 1.** AI Metro overview: input, dynamic edge cost, A* search and route output.

## II. PROPOSED SYSTEM

AI Metro models the metro network as a graph and finds the route of minimum generalised cost using A* search. The cost of a segment is built from four parts: the fare, the travel time, a crowd penalty and a transfer penalty. Availability data is applied first, so closed stations and connections never enter the search. The system then presents the best route, a trip summary and one or two alternatives.

### *A. System Architecture*

The architecture has seven modules (Fig. 1). The colour scheme is blue for user input and output, green for the data modules, yellow for processing and red for the AI algorithm.

- User Input: source station, destination station and an optional preference (Balanced, Fastest, Least crowded or Fewest transfers).
- Metro Network Database: stations, lines, distances, fares and running times.
- Crowd Information Module: a Low, Medium or High level for each station and segment.
- Station Availability Module: the status of each station and connection (OPEN, DELAYED, CLOSED, MAINTENANCE or DISRUPTED).
- Dynamic Cost Calculation: computes edge costs from the current crowd and availability data and removes unavailable elements.
- A* Search Engine: finds the optimal path, guided by the heuristic h(n), and returns alternatives.
- Route Recommendation: shows the route, travel time, fare, crowd summary, number of transfers and alternatives.

The data flows from the user to the three data modules, then to the dynamic cost calculation, the A* search and the route recommendation, which is returned to the user.

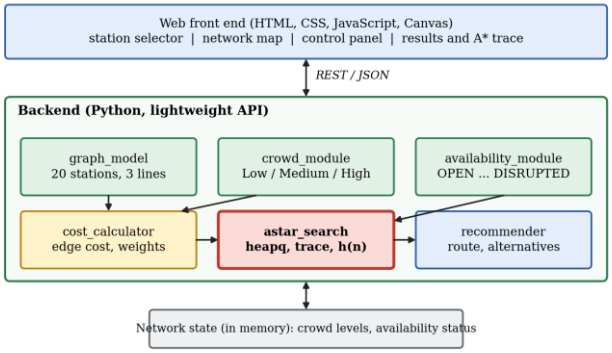



**Fig. 2.** AI Metro architecture: front end, backend modules and network state.

### *B. Problem Formulation*

The network is an undirected graph G = (V, E). Each node is a station, and each edge is a track segment between two adjacent stations on one line, so travel is possible in both directions. An interchange station belongs to more than one line.

Because a transfer penalty applies when the passenger changes line, the cost of an edge depends on the line used to arrive. A* therefore searches over states (station, line) rather than stations alone. The start state is (source, none), so the passenger can board any line at the source without penalty. A state is a goal if its station is the destination. This keeps the cost of every move dependent only on the current state, which A* needs in order to stay optimal.

### *C. Edge Cost Function*

The cost of an edge is

edge_cost = travel_cost + time_cost + crowd_penalty + transfer_penalty

and the cost of a path is the sum of its edge costs. Table 1 gives the components and default values.

**TABLE I** Components of the edge cost

| Component | Definition | Default value |
|---|---|---|
| travel_cost | Fare of the segment, proportional to its length | Rs 2.0 per km × w_far |
| time_cost | Running time plus dwell time, plus extra minutes if the segment is DELAYED | 1.5 min/km + 1.0 min dwell (+6 min if delayed) × w_time |
| crowd_penalty | Penalty for the higher of the segment's own level and the level of the station entered | Low = 0, Medium = 4, High = 10, × w_crow |
| transfer_penalty | Added only when the line used differs from the line used to arrive | 8 × w_transfer |

The weights w fare, w time, w crowd and w transfer default to 1 and are reset by the user's preference. Four presets are provided: Balanced (1, 1, 1, 1), Fastest (0.5, 3, 0.3, 0.5), Least crowded (0.5, 0.7, 4, 1) and Fewest transfers (0.5, 0.7, 0.7, 4). All weights are non-negative, so every edge cost is non-negative, as A* requires. In the reported trip time, each transfer adds 4 minutes of walking and waiting.

### *D. Heuristic Function*

The heuristic estimates the remaining cost from a state n to the destination using the straight-line distance d(n, goal) in kilometres:

$$h(n) = k \times d(n, \text{goal})$$

Here k is the smallest value, over all usable edges e, of (travel_cost(e) + time_cost(e)) divided by the straight-line length of e. Scaling by k puts the heuristic in the same units as g(n). In the sample network k = 4.137 cost units per km under the Balanced preset.

Admissibility. Any path from n to the goal consists of edges $e_1, \ldots, e_m$. The cost of each edge is at least k times its straight-line length, because the crowd and transfer penalties are never negative. By the triangle inequality, the sum of these lengths is at least d(n, goal). The true remaining cost is therefore at least k × d(n, goal), so h never over-estimates.

Consistency. For any edge (u, v), $h(u) - h(v) \leq k \times d(u, v) \leq \text{cost}(u, v)$. A state therefore never needs to be re-opened after it is closed. When an unavailable edge is removed, k is recomputed over the remaining edges, so the heuristic always matches the filtered graph.

*E. A* Algorithm*

A* keeps an open set of generated states waiting to be expanded, held in a priority queue ordered by f(n) = g(n) + h(n), and a closed set of states already expanded. At each step the state with the lowest f is removed from the open set and added to the closed set. If its station is the destination, the search stops and the path is rebuilt through parent links. Otherwise each usable neighbouring edge is relaxed: its cost (travel, time and crowd, plus the transfer penalty if the line changes) is added to g, and the neighbouring state is added to the open set with f = g + h if this gives a lower cost than before. If the open set becomes empty, no route is available.

The implementation is an explicit loop over Python's heapq priority queue, not a library call. It records every expansion (step, station, line, g, h, f and open-set size). This trace is shown in the demonstration screen and the console output to explain how A* works.

*F. Availability Filtering and Alternative Routes*

Availability. Each station and connection has a status: OPEN, DELAYED, CLOSED, MAINTENANCE or DISRUPTED. The last three statuses are hard constraints, so the element is excluded from the graph before A* runs and is never merely penalised. A DELAYED element stays in the graph, but its time cost grows by a fixed number of minutes. Statuses are looked up whenever an edge is considered, so they can be toggled at run time and the next query uses the new state without restarting the application.

Alternatives. After the best route is found, each segment of that route is banned in turn and A* is re-run. Every resulting route that differs from those already found and costs no more than 1.6 times the best cost is a candidate. The two cheapest candidates are shown as alternatives. This is a simplified form of the edge-removal idea in Yen's K-shortest-paths method.

*G. Implementation Overview*

The prototype has one component per module: graph_model, crowd_module, availability_module, cost_calculator, astar_search and the recommender, plus a web front end. The front end has a station selector and preference control, a network view with crowd-coloured stations and a highlighted path, a control panel for changing crowd levels and closing stations, and a results panel with an A* trace table. Crowd and availability data are simulated.

## III. EXPERIMENTAL SETUP AND RESULTS

*A. Experimental Setup*

Implementation. The search engine is written in Python 3 using only the standard library (heapq, math, itertools and dataclasses). It is about 200 lines long and is divided into the modules of Section 2. The same engine can be exposed through a lightweight API to the web front end.

Dataset. The seeded sample network has 20 stations, 3 lines, 21 segments and 4 interchange stations (Table 2). Distances are 1.1 times the straight-line length of each segment. Fares and times follow Table 1. The four interchange stations are set to Medium crowd at start-up to imitate peak hours. Crowd and availability data are simulated, and no real transit data are used.

**TABLE II** Seeded sample network

| Line | Stations | Terminals | Interchanges |
|---|---|---|---|
| Blue | 9 | Airport – Central | Guindy (Red), Nandanam (Green), Central (Red) |
| Green | 7 | Koyambedu – Thiruvanmiyur | Koyambedu (Red), Nandanam (Blue) |
| Red | 8 | Central – Guindy | Central (Blue), Koyambedu (Green), Guindy (Blue) |

The Red and Blue lines form a ring through Guindy, Koyambedu and Central, so most station pairs have more than one reasonable route. This lets crowd and closures change the answer.

Evaluation methods. Three kinds of evaluation were carried out:

- Scenario tests check whether the recommended route changes in the expected direction when crowd, availability or preferences change.
- A correctness check compares A* with Dijkstra's algorithm (the same code with h = 0), which is known to be optimal. It covers every station pair under normal conditions and 200 random configurations of crowd levels, closures and preferences (30 random queries each, random seed 42).
- An efficiency measurement counts the nodes expanded by each algorithm.

*B. Scenario 1: A* Trace for a Multi-Hop Trip*

A passenger travels from Airport to Thiruvanmiyur under normal peak-hour crowd and the Balanced preference. The recommended route is Airport – Meenambakkam – Guindy – Little Mount – Saidapet – Nandanam (change from Blue to Green) – Mylapore – Adyar – Thiruvanmiyur. It is 15.6 km long, takes 35 minutes, costs Rs 31 and has one transfer, with 6 Low and 2 Medium crowd segments. The total cost is 78.70, and no alternative falls within the 1.6 times tolerance. Table 3 shows the A* expansion trace.

**TABLE III** A* expansion trace for Airport to Thiruvanmiyur

| Step | State expanded (station, line) | g(n) | h(n) | f(n) | Open set after removal |
|---|---|---|---|---|---|
| 1 | Airport (start) | 0.00 | 43.44 | 43.44 | 0 |
| 2 | Meenambakkam (Blue) | 7.02 | 38.70 | 45.72 | 0 |
| 3 | Guindy (Blue) | 21.05 | 32.71 | 53.76 | 1 |
| 4 | Little Mount (Blue) | 27.40 | 29.23 | 56.63 | 2 |
| 5 | Airport (Blue) | 14.04 | 43.44 | 57.48 | 2 |
| 6 | Saidapet (Blue) | 32.88 | 26.98 | 59.86 | 1 |
| 7 | Nandanam (Blue) | 43.55 | 24.91 | 68.46 | 1 |
| 8 | Mylapore (Green) | 61.16 | 16.68 | 77.84 | 3 |
| 9 | Adyar (Green) | 70.77 | 7.46 | 78.23 | 4 |
| 10 | Thiruvanmiyur (Green) | 78.70 | 0.00 | 78.70 | 4 |

The f value never decreases from one expansion to the next, which reflects the consistency of h. At step 5 the state (Airport, Blue) is expanded because the state space includes the line, so a station can appear again with a different line. This expansion leads nowhere better and the search moves on. At step 8 the cost rises from g = 43.55 at Nandanam to g = 61.16 at Mylapore, and 8.00 of that increase is the transfer penalty for changing from Blue to Green. A* expanded 10 states and generated 13, whereas Dijkstra's algorithm expanded 21 states and returned the same cost of 78.70.

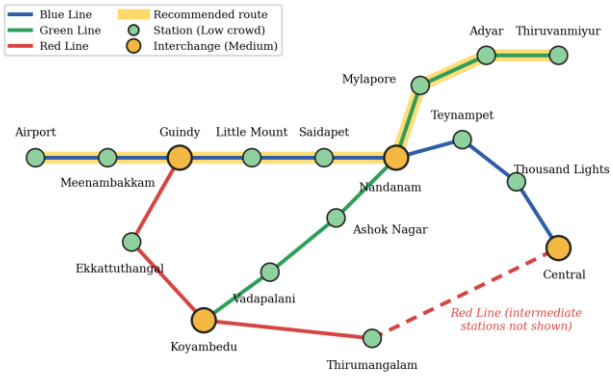



**Fig. 3.** Figure 3.

*C. Scenario 2: Effect of Crowd Level*

A passenger travels from Koyambedu to Thousand Lights. Under normal conditions the best route follows the Green Line to Nandanam, changes to the Blue Line and continues to Thousand Lights. The alternative uses the Red Line to Central and then the Blue Line. Heavy crowding is then simulated by setting Vadapalani, Ashok Nagar, Nandanam and the segment between Ashok Nagar and Nandanam to High. The recommendation switches to the Red Line route, although it is 1.1 km and 2 minutes longer. The Green route is now shown only as an alternative (Table 4).

**TABLE IV** Effect of crowd on the route from Koyambedu to Thousand Lights

| Condition | Route (transfer station) | Time (min) | Fare (Rs) | Transfers | Crowd | Cost |
|---|---|---|---|---|---|---|
| Normal, best | Green + Blue (Nandanam) | 26 | 22 | 1 | Low | 55.89 |
| Normal, alternative | Red + Blue (Central) | 28 | 24 | 1 | Low | 60.77 |
| Heavy crowd, best | Red + Blue (Central) | 28 | 24 | 1 | Low | 60.77 |
| Heavy crowd, alternative | Green + Blue (Nandanam) | 26 | 22 | 1 | High | 81.89 |

The route that is fastest on paper is no longer the best once crowd is priced in. Its raw time and fare are unchanged, and only its crowd penalty has risen. A distance-only planner cannot show this behaviour.

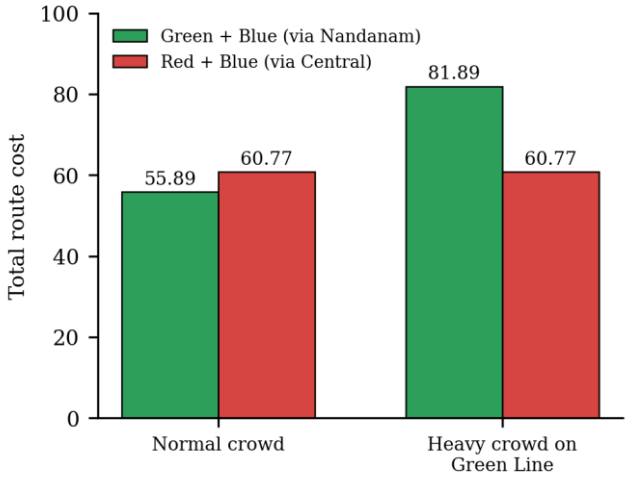



**Fig. 4.** Figure 4.

*D. Scenario 3: Effect of Delays, Closures and Maintenance*

A passenger travels from Koyambedu to Teynampet. Table 5 compares four situations.

**TABLE V** Effect of availability on the recommended route

| Condition | Route | Time (min) | Fare (Rs) | Transfers | Cost |
|---|---|---|---|---|---|
| All open | Green + Blue via Ashok Nagar, Nandanam | 22 | 19 | 1 | 48.87 |
| Ashok Nagar DELAYED | Same route (+6 min delay) | 34 | 19 | 19 | 60.87 |
| Ashok Nagar CLOSED | Red + Blue via Thirumangalam … Central, Thousand Lights | 28 | 1 | 28 | 67.79 |

A delayed station stays in the graph and the route is kept, because the detour (cost 67.79) is still more expensive than waiting (cost 60.87). A closed station is removed completely, so the planner reroutes around it and offers the next-best route as an alternative.

Two further tests used maintenance at Nandanam, an interchange. For Airport to Teynampet, the normal route is a direct Blue Line trip of 23 minutes and Rs 22 with no transfer. With Nandanam under maintenance, the planner returns a 57-minute, Rs 51 route with two transfers that goes around the Red Line ring. For Koyambedu to Adyar, no route exists, because Mylapore, Adyar and Thiruvanmiyur are reachable only through Nandanam. The system correctly reports that no route is available instead of returning an invalid path.

### *E. Scenario 4: Effect of User Preferences*

A passenger travels from Koyambedu to Guindy while the Red Line segment between Koyambedu and Ekkattuthangal and the station Ekkattuthangal are set to High. Table 6 shows the result for each preference.

**TABLE VI** Route chosen for each preference (Koyambedu to Guindy)

| Preference | Route | Time (min) | Fare (Rs) | Transfers | Crowd segments (L/M/H) | Nodes expanded |
|---|---|---|---|---|---|---|
| Balanced | Red, direct | 13 | 14 | 0 | | |
| Fastest | Red, direct | 13 | 14 | 0 | | |
| Least crowded | Green + Blue via Nandanam | 27 | 23 | 1 | | |
| Fewest transfers | Red, direct | 13 | 14 | 0 | | |

The direct Red Line trip is recommended by the Balanced, Fastest and Fewest transfers presets. The Least crowded preset instead chooses the Green and Blue trip through Nandanam, which takes 14 extra minutes but avoids the crowded Red Line section. Costs are not compared across presets, because each preset uses different weights.

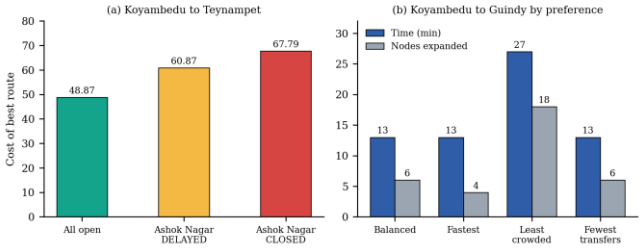



**Fig. 5.** Figure 5.

### *F. Correctness and Efficiency of A**

Table 7 compares A* with Dijkstra's algorithm. Under normal conditions all 380 ordered station pairs were tested. In the randomised test, 6,000 queries were run on 200 random configurations. In each configuration every station's crowd level was drawn at random, two stations were closed, put under maintenance or marked disrupted, and one of the four presets was chosen. Of these queries, 1,956 had no route (both algorithms agreed on this) and 4,044 had a route.

**TABLE VII** A* versus Dijkstra's algorithm

| Experiment | Routes compared | Cost mismatches | A* nodes expanded (mean) | Dijkstra nodes expanded (mean) | Reduction |
|---|---|---|---|---|---|
| Normal crowd, all station pairs | 380 | 0 | 7,34 | 12.78 | 42..5% |
| Random crowd, closures and presets | 4,044 | 0 | 8,59 | 11.10 | 22.7% |

In all 4,424 comparisons A* returned exactly the same cost as Dijkstra's algorithm, which supports the admissibility argument of Section 2.4. A* expanded fewer nodes in both experiments. The saving is smaller in the random experiment because crowd penalties and closures make the real remaining cost larger than the straight-line estimate, so the heuristic is less tight. The sample network is small, so the absolute numbers are small, and the percentage reduction is the meaningful result.

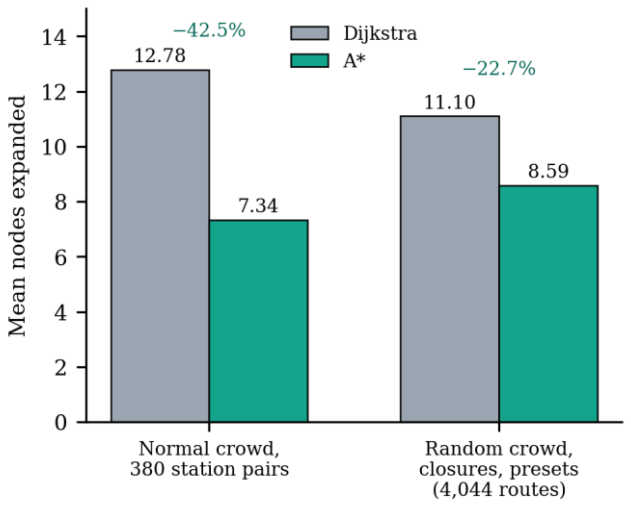



**Fig. 6.** Figure 6.

*G. Discussion*

- The recommended route is the minimum-cost route under the chosen cost function, not necessarily the shortest in km or minutes.
- Crowd levels move the recommendation only when a plausible alternative exists within a reasonable cost. Otherwise the planner keeps the original route.
- Hard exclusion of unavailable elements guarantees that a closed station is never recommended and that unreachable destinations are reported.
- Preferences re-weight the same formula, so the user can trade time against comfort without changing the algorithm.
- The trace of expanded states, with g, h and f values, makes every recommendation explainable.

Limitations. The network is a small, schematic sample. Crowd and availability data are simulated. The fare is a simple distance-proportional model rather than a real fare table. The penalty values (4 and 10 for crowd, 8 for a transfer) were chosen for demonstration and were not calibrated against passenger behaviour.

## IV. CONCLUSION AND FUTURE WORK

This paper presented AI Metro, a crowd-aware metro route planner based on A* search. The network is a weighted graph, and the edge cost combines fare, time, crowd and transfer penalties. Unavailable stations and connections are removed before the search, and an admissible, consistent straight-line heuristic keeps the result optimal. The A* implementation is written explicitly and exposes its open and closed sets and f, g and h values, so it can be explained. Tests on a 20-station, 3-line sample network showed that A* matched Dijkstra's algorithm in all 4,424 comparisons while expanding fewer nodes. They also showed that the recommended route reacts correctly to crowding, delays, closures and user preferences.

The prototype can be extended in the following directions:

- Replace the simulated crowd data with live or historical passenger-load data, for example from smart-card taps or platform sensors.
- Use a real fare table, real timetables and train frequencies, so that waiting time at transfers is modelled instead of a fixed value.
- Learn the crowd and transfer penalties from passenger choice data instead of fixing them by hand.
- Predict crowd levels for the planned time of travel with a machine-learning model and use the prediction in the cost function.
- Replace the simplified alternative-route method with Yen's full K-shortest-paths algorithm, and support multi-modal journeys with buses and walking.

## V. REFERENCES

1. Bast, H., Delling, D., Goldberg, A., Müller-Hannemann, M., Pajor, T., Sanders, P., Wagner, D. and Werneck, R.F. (2016) 'Route Planning in Transportation Networks', in Algorithm Engineering, Lecture Notes in Computer Science, Vol. 9220, Springer, pp. 19–80.
2. Dijkstra, E.W. (1959) 'A note on two problems in connexion with graphs', Numerische Mathematik, Vol. 1, No. 1, pp. 269–271.
3. Hart, P.E., Nilsson, N.J. and Raphael, B. (1968) 'A Formal Basis for the Heuristic Determination of Minimum Cost Paths', IEEE Transactions on Systems Science and Cybernetics, Vol. 4, No. 2, pp. 100–107.
4. Wardman, M. and Whelan, G. (2011) 'Twenty years of rail crowding valuation studies: Evidence and lessons from British experience', Transport Reviews, Vol. 31, No. 3, pp. 379–398.
5. Yen, J.Y. (1971) 'Finding the K shortest loopless paths in a network', Management Science, Vol. 17, No. 11, pp. 712–716.